



Sommaire

Intelligence Artificielle - IA

TP02 - Python - Matplotlib



Dr. Mohamed Ould Djibril

November 20, 2023

Département d'Informatique et Méthodes Quantitatives
Institut Supérieur de Comptabilité et d'Administration
des Entreprises - ISCAE

mohamed.djibril@gmail.com

1 Matplotlib

Présentation

Éléments de base

Tracé d'une fonction

Fonctions à une seule variable

Regalements

Les TP sont notés

- Suivre les instructions ci-dessous étape par étape.
- Exécuter les codes et corriger les exercices dans un cahier Jupyter qui porte le nom : [VotrelD_IA_TP02.ipynb](#).
- Répondre aux questions et compléter les codes en Salle de classe ou la Maison.
- Envoyer le fichier par mail à : mohamed.djibril@iscae.mr.
- Date du rendu sera communiquée en cours.

Matplotlib

Présentation:

- Il s'agit de l'une des bibliothèques python les plus utilisées pour représenter des graphiques en 2D.
- Elle permet de produire une grande variété de graphiques de grande qualité.
- Le module **pyplot** de **matplotlib** est l'un de ses principaux modules.
- le module **pyplot** regroupe un grand nombre de fonctions qui servent à créer des graphiques et les personnaliser .
- Pour importer le module **pyplot** de **matplotlib** en tant que **plt**, on place donc cette ligne au début de notre fichier.

```
import matplotlib.pyplot as plt
```

Eléments de base



Le plus important dans une figure est **figure**.

Une figure est créée à l'aide de la méthode `figure()`

- On peut spécifier sa taille.
- On peut spécifier sa couleur d'arrière-plan: **facecolor**
- On peut spécifier son titre et un sous titre : `title`, `subtitle`
- La couleur de l'arrière-plan ne sera pas enregistré lors du sauvegarde du figure dans un fichier.
- C'est la couleur de fonction **savefig** qui sera retournée.
- Si pas besoin de couleur du fond : **transparent=True** dans **savefig**.

5 / 24

Eléments de base

Axes:

- **Axes** est le second élément le plus important.
- Il correspond à l'espace actuel où vos données seront représentées.
- Il est aussi appelé **subplot**.
- Vous pouvez avoir un ou plusieurs **axes** par **figure**.

6 / 24

Eléments de base

Axes:

- Chaque axe est un cadre de quatre cotés (left, top, right et bottom).
- Les cotés sont appelés les **Spines** (epines).
- Chacun des **spines** peut être décoré avec deux types de cochers (**ticks**): un petit et un grand formant ainsi une règle.
- Les cochers peuvent pointer à l'intérieur comme à l'extérieur.
- Chacun des **spines** peut être décoré avec : un **ticks label** et **label**.
- Par défaut matplotlib décore uniquement : le spine d'un bas et le spine et celui d'à gauche.

Eléments de base

Axis:

- Les spines décorés sont appelés **axis**, axe au sens mathématiques.
- L'axe horizontal est appelé **xaxis**.
- L'axe vertical est appelé **yaxis**.
- Chacun des spines (ou règles/axes), possède des cochers (petit et grand) avec leurs labels et un label (axis label).
- **Pyplot**, utilise le concept de la **figure active** et le **cadre actif**. C'est à dire qu'une tout appel d'une fonction de pyplot sera appliqué au cadre actif de la figure active.

Eléments de base

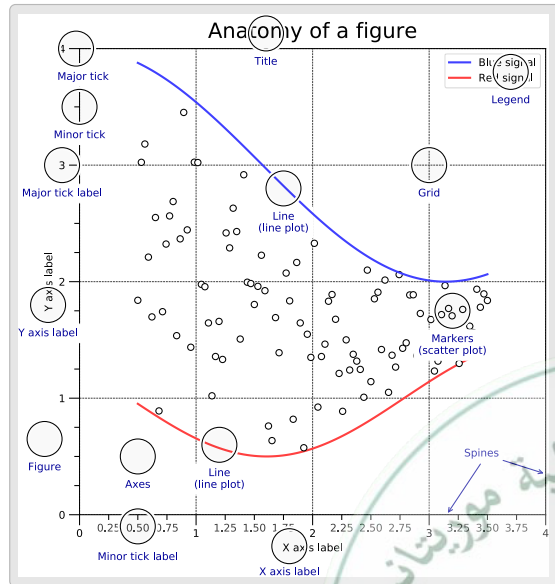


Exercice :

Identifier et nommer dans une cellule chaque élément dans la figure en face.

Documentation :

<https://www.matplotlib.org>



9 / 24

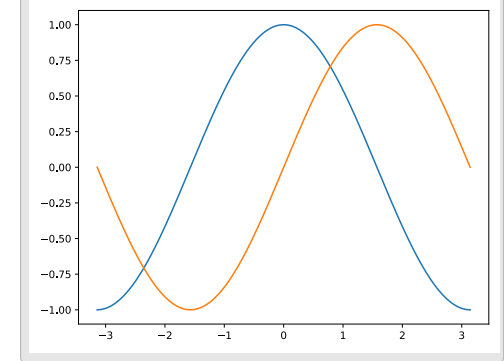
Eléments de base

Exemple :

```
import numpy as np
import matplotlib.pyplot as plt
pi = np.pi
X = np.linspace(-pi, pi, 257)
C, S = np.cos(X), np.sin(X)
plt.plot(X, C)
plt.plot(X, S)

plt.tight_layout()
plt.savefig("matplotlib_ex1.pdf")
plt.show()
```

Reprendre l'exemple pr obtenir



10 / 24

Styles de programmation Matplotlib

Styles de programmation Matplotlib

- Le style 'pyplot' qui utilise directement des fonctions du module pyplot, il est simple et pratique.
- Le style 'Orienté Objet' qui est complet et recommandé par la documentation de Matplotlib.

Styles de programmation Matplotlib

Version pyplot

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 30)
y = np.cos(x)

plt.plot(x, y)
plt.ylim(-2, 2)

plt.show()
```

Version Orienté Objet

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 30)
y = np.cos(x)

fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_ylim(-2, 2)
plt.show()
```

Pyplot un vecteur



- Mise en œuvre :
- Soit un vecteur y , pyplot va tracer les lignes reliant les points $A(0, a)$, $B(1, b)$ et $C(2, c)$.
 - En fait, y est considéré par pyplot comme les ordonnées et les abscisses sont automatiquement générées.

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4])
plt.ylabel('some numbers')
plt.show()
```

Exercice :

- Executer le code
- Constater que pyplot a considéré les points du vecteur fournit $y = (1 \ 2 \ 3 \ 4)$ comme les ordonnées et il a généré automatiquement le vecteur des abscisses $x = (0 \ 1 \ 2 \ 3)$
- Remplacer `plt.plot([1, 2, 3, 4])` dans une nouvelle cellule par `plt.plot([0, 2, 3, 4], [1, 2, 3, 4])` quelle est la différence?

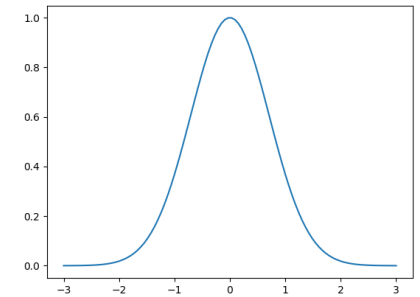
Graphes d'une fonction à une seule variable

Exemple :

Tracer la fonction $f : [-3, 3] \rightarrow \mathbb{R}$ définie par $f(x) = e^{-x^2}$:

```
import numpy as np
import matplotlib.pyplot as plt
def f(x):
    return np.exp(-x**2)
a,b = -3,3
X = np.linspace(a,b,num=100)
Y = f(X)
plt.plot(X,Y)
plt.show()
```

Reprendre l'exemple pr obtenir :



13 / 24

14 / 24

Graphes d'une fonction à une seule variable

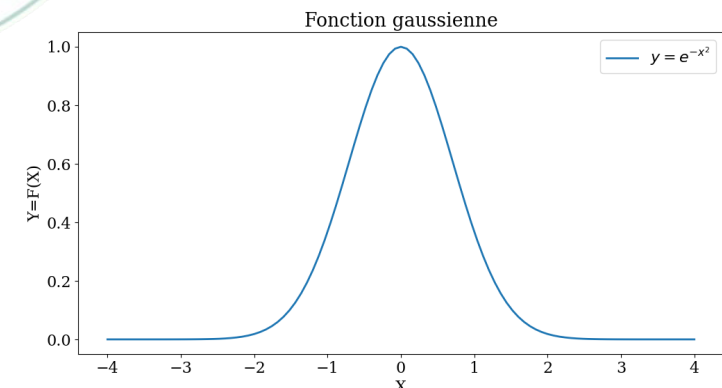
Graphes d'une fonction à une seule variable

Améliorations:

Pour avoir un résultat de qualité, il faut rajouter un titre lisible, des labels sur les axes et sur les courbes.

```
plt.figure(figsize=(10,6))
plt.plot(X,Y,lw=2,label="$y=e^{-x^2}$")
# légendes
plt.title("Fonction gaussienne")
plt.xlabel("X")
plt.ylabel("Y=F(X)")
# label
plt.legend();
# sauvegarde du tracé dans un fichier
plt.savefig("courbe.png")
```

Reprendre l'exemple précédent pour obtenir ce résultat :



15 / 24

16 / 24

Graphe d'une fonction à une seule variable



Deux vecteurs :

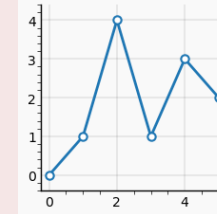
```
X = np.array([0,1,2,3,4,5])
```

```
Y = np.array([0,1,4,1,3,2])
```

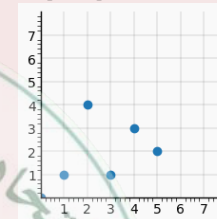
- La commande `plt.plot(X,Y)` affiche les points (x,y) pour x parcourant X et y parcourant Y et les relie entre eux.
- `plt` est le nom raccourci que nous avons donné au sous-module python `pyplot` du module `matplotlib`.
- Lorsque les points sont suffisamment rapprochés cela donne l'impression d'une courbe lisse.
- Il existe d'autres commandes similaires `.plot(X,Y)` pour représenter (X, Y).
- Consulter la documentation officielle pour les découvrir.

Graphe d'une fonction à une seule variable

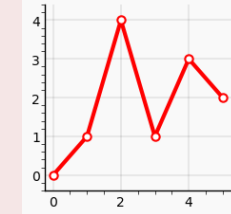
Utiliser les données de l'exemple pour reproduire :



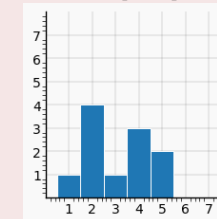
`plt.plot(X,Y)`



`plt.scatter(X,Y)`



`plt.plot(X,Y,linewidth=3,color='red')`



`plt.bar(X,Y)`

17 / 24

18 / 24

Graphe d'une fonction à une seule variable

Tracés de plusieurs fonctions:

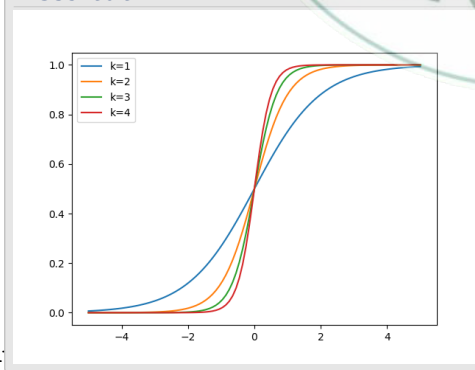
On peut effectuer plusieurs tracés sur le même dessin. Ici des fonctions sigmoïdes

$$f_k(x) = \frac{1}{1 + e^{-kx}}$$

avec un paramètre k qui varie.

```
def f(x,k):
    return 1/(1+np.exp(-k*x))
a,b = -5,5
X = np.linspace(a,b,num=100)
for k in range(1,5):
    Y = f(X,k)
    plt.plot(X,Y, label="k={}".format(k))
plt.legend()
plt.show()
```

Résultat :



La fonction subplot() :

- La fonction `subplot()` permet d'organiser différents tracés à l'intérieur d'une grille d'affichage.
- Il faut spécifier le nombre de **lignes**, le nombre de **colonnes** ainsi que le **numéro du tracé**.

Reprendre le code suivant:

```
import numpy as np
import matplotlib.pyplot as plt
def f(t):
    return np.exp(-t) * np.cos(2*np.pi*t)

t1 = np.arange(0.0, 5.0, 0.1)
t2 = np.arange(0.0, 5.0, 0.02)
fig, axs = plt.subplots(2, 1)
axs[0].plot(t1, f(t1), "bo")
axs[0].plot(t2, f(t2), "k")
axs[1].plot(t2, np.cos(2*np.pi*t2), "r--")
plt.show()
```

19 / 24

20 / 24

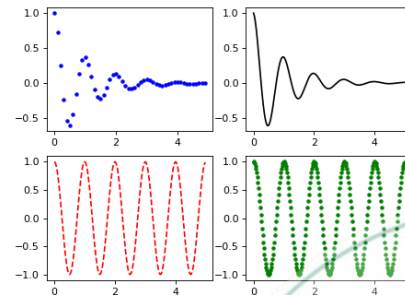
La fonction subplot()



Tracer de plusieurs fonctions:

- 1 En s'inspirant de l'exemple précédent, écrire le code qui permet de donner la sortie en face.
- 2 Utiliser la documentation pour changer le style de la tracée et la couleur de chaque fonction.

Résultat attendu:



Graphes d'une fonction à deux variables

Principe :

- 1 On calcule d'abord une grille (X, Y) de points à l'aide meshgrid().
- 2 On calcule les valeurs Z de la fonction sur cette grille.
- 3 Ensuite le tracé du graphe se fait grâce à l'instruction plot_surface(X, Y, Z).
- 4 Augmenter n, la grille alors est plus dense, observer maintenant la surface dessinée.
- 5 Tourner la surface pour mieux la visualiser: ax.view_init(40,-30)

Executer le code :

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

n = 5
VX = np.linspace(-2.0, 2.0, n)
VY = np.linspace(-2.0, 2.0, n)
X,Y = np.meshgrid(VX, VY)
def f(x,y):
    return x**2-y**2
Z = f(X,Y)
fig = plt.figure()
ax = plt.axes(projection='3d')
ax.view_init(40, -30)
ax.plot_surface(X, Y, Z)
plt.show()
```

21 / 24

22 / 24

Graphes d'une fonction à deux variables

Cours suivant

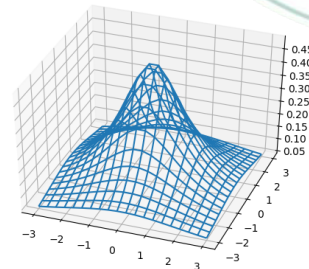
Exercice:

- 1 En s'inspirant de l'exemple précédent, utiliser plot_wireframe(X, Y, Z) au lieu de plot_surface(X, Y, Z) pour tracer :

$$f(x,y) = \frac{1}{2 + x^2 + y^2}$$

- 2 Diminuer n pour et executer.

Résultat attendu



Nous allons étudier :

- **Apprentissage d'un perceptron :**
Implementation de l'algorithme d'apprentissage d'un Perceptron pour la classification.
- **Multi-couches de neurones :**
présentation et mise en équation
- **Fonctions d'activation :**
étudier les différents types des fonctions utilisées